

University of Florence

Master's degree in Software: Science and Technology

Academic Year 2025/2026

Automated Software Testing

Author: Francesco Biribò

Student ID: 7199598

Link to GitHub repository: https://github.com/whocaresleft/hospital-scheduler

Contents

1 Description **1**

2 Design choices **2**
2.1 Model 2
2.2 Architecture stack 3
2.3 Transactions 5
2.4 App module 6

3 Encountered problems and solution **6**
3.1 Version incompatibilities 6
3.2 Mutation tests timeout 7
3.3 Wayland UI tests 7
3.4 LGoodDatePicker non-determinism in UI tests 9
3.5 Development environment 10

4 Used technologies **11**
4.1 High rating technologies 11
4.2 Other new technologies 14

5 Building instructions **16**
5.1 Building in Linux 16

6 Running instruction **18**
6.1 Running natively 18
6.2 Running in Docker (Linux only) 18
6.3 Manual execution 19

1 Description

This project implements an Hospital shifts management applicationm between doctors and hospital departments. A **Doctor** is an hospital worker (well, the name Doctor is a bit specific, it could be expanded in the future, the main idea started just from that), represented by an Id, first name, and last name. A **Department** is a section of the Hospital in which a Doctor could be found working (e.g. Emergency Room or Surgery Room), it's represented by an Id and a name. Finally, a **Shift** represents a time slot, within a specific day, in which a Doctor is scheduled to work in a certain Department (e.g. doc1-er, 26/08/2026, 08:00-15:00).

The application was developed with a MVP architecture with transaction support in mind. The overall stack is composed of four layers: view, presentation, transactional, and data. The view layer consists of three views, one for each entity, defined as interfaces. The implementation of such views was done in Swing, as each view is a panel which can be integrated, for example, in a frame. This Swing implementation can also interact with the presentation layer, to issue commands. In the presentation layer there is a presenter for each entity. On one side, this layer interacts with the data layer (however this is done through the transactional layer) to perform CRUD operations regarding the corresponding entity. On the other side, it interacts with the view layer in order to update the view, and show the outcome of the performed operation. While each presenter only works with one view (because they both handle only one entity), its not entirely true that it works with a single repository only. The reason lies in the n-n relation between Doctors and Departments, which results in a third entity, Shift. The relation between Doctors, Shifts, and Departments is 1-n-1, this the following two must be considered:

- Deleting either a Doctor or Department requires also handling the corresponding Shifts (DoctorPresenter and DepartmentPresenter ALSO work with the Shift repository);
- Inserting (or updating) a Shift requires consistency with the currently present Doctors and Departments (ShiftPresenter works with all three repositories).

The transactional layer is what makes multi-repository operations possible, by encapsulating the presenter's operations in a transaction block (without the presenter needing to perform extra actions). The data layer is made up of the three repositories, one for each entity. These were defined as interfaces, and have an implementation both in MongoDB and MariaDB (this means the repositories are created to work with a MongoDB, or MariaDB, server respectively).

The transactional layer as well was implemented to work both with MongoDB and MariaDB. As expected, a TransactionManager implemented for a particular database requires the repositories to be of that same "type" (as in, developed for the same database). More information on the actual implementation will be given in the next section.

2 Design choices

I decided to create this project as a multi module maven project. As I already had an MVP (or MVC) architecture in mind from the start, I thought about partitioning the various packages, and classes in multiple modules according to their scope. The various modules are explained in the README.md file of the repository. In this report I will mainly focus on the modules `app`, `domain`, and `presentation`.

2.1 Model

The domain model of this application consists of three entities. There were just two originally, Doctor and Department, in a many to many relation, (such relation generally translates to a new table) which became the third entity, Shift. These entities are particularly simple, consisting only of a few non-complex parameters. Doctor represents an hospital worker (e.g. a doctor) with a first and last name, identified by an id. The second domain entity is Department, which represents a section, or structure, within the hospital in which a Doctor can be found working (examples include an ER or a surgery room), or even an abstract concept such as 'Appointment'. In the end, it only has a name and is identified by an id as well.

The third entity, which started out as a relation, is Shift, which represents a work shift that a Doctor works in a certain Department. I also added other fields to differentiate shifts between the same doctor and department. For that reason, I added three fields, one `LocalDate` and two `LocalTime`s, which represent the time slot, within a specific day, in which the work shift takes place. The Shift is identified by the quintuple[1].

As a final touch, I decided not to use a simple String to store the id of doctors and departments, but rather a class `Id`, whose instances just store a String internally. However such string is checked at creation to make sure only valid ids can be used, that is, ids containing only letters, digits, or underscores.

Each entity (and the Id) was implemented with a private constructor and final fields, exposing only a static factory method for creation and getter methods. This choice was to make sure each entity was seen by other developers as just an ephemeral domain object: they can be created but not modified, just to be used within the MVP architecture, while the actual updates are performed on the records inside the database.

I put all three domain model classes inside the module **domain** and in the **.model** package[2]. Similarly, since each of these depend on the Id class, while not being a domain model object, I chose to place it in the same module, under the **.core** package.

[1] When implementing the repositories I had to add an extra field to use as primary key (in mariadb) or `_id` (in mongodb). I defined an helper method (in each shift repository) to generate such string id, by concatenating the string values of the five fields, making sure the Shift is practically identified by the quintuple.

[2] Each package has the same root **org.duckdns.whocaresleft**

2.2 Architecture stack

The implemented MVP architecture has four layers, which are view, presentation, transactional, and data. While transactions will be explained in the next subsection, this one will keep focus on the remaining three.

Figure 1 shows how the layers interact with each other. Each entity has its own repos-

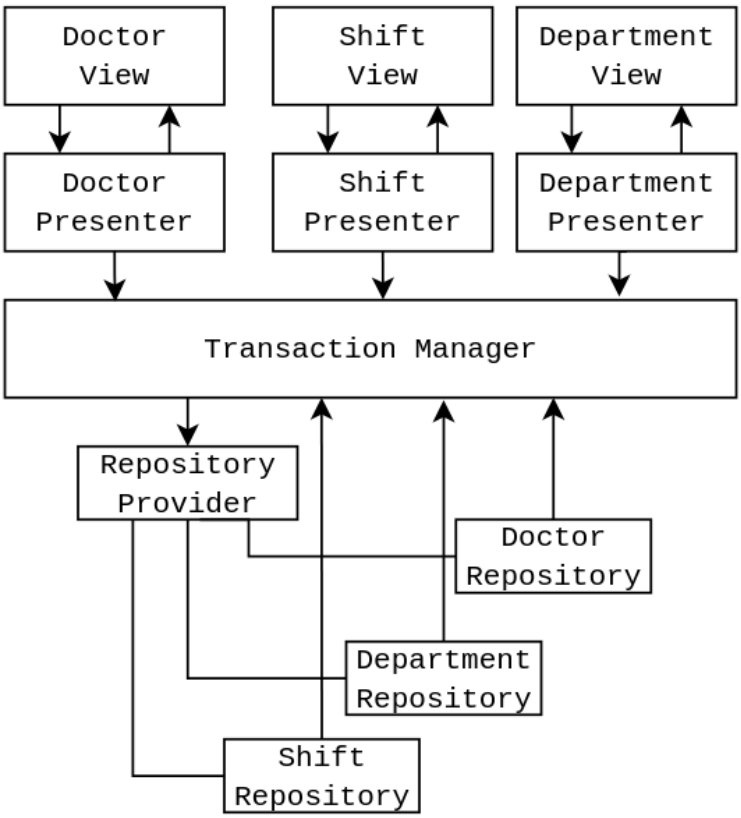


Figure 1: MVP with transaction suppport stack

itory, which only know how to perform CRUD operation regarding that entity. Each entity also has its own view, which knows how to present it to a user. Finally, each entity has its own presenter, which orchestrates the operations on the repositories, and shows them on the corresponding view.

Each view has its own presenter (and vice versa), since each view only concerns about a single entity, operations requested (e.g. by GUI buttons) by one view can only be about its corresponding entity, so one presenter is sufficient. The same is not true for the relation between presenters and repositories, but why? Simply because of the many to many relation: deleting either a doctor or department has an effect on the corresponding shifts, and adding a shift requires both the doctor and department to be present.

However, presenters don't directly interact with the repositories, or to say it better: each presenter does not know (or have a reference to) the repositories directly. Each com-

municates with a transaction manager, whose purpose is to wrap repositories operations inside a single transaction (note that this is required since presenters work on multiple repositories, even for a single 'high level' operation).

Presenters can work with the various repositories, through the transaction manager, thanks to the `RepositoryProvider` interface, which only has three methods, `getDoctortRepository()`, `getDepartmentRepository()`, and `getShiftRepository()`, which the presenter can use to retrieve the needed repositories directly during the operation.

Repositories

Repositories were initially defined as interfaces with just CRUD methods for the corresponding entity. I then decided to implement all three repositories both 'with' MongoDB and MariaDB (which means, using an API that lets the repository communicate with a MongoDB and MariaDB server respectively).

Each repository (for each entity) requires to define the following methods: `save`, `delete`, `update`, `findAll`, and `findById`. While the latter is only for the doctor and department repositories, the shift repository has its variants, `findByDoctorId` and `findByDepartmentId`, which both return a list of shifts.

Repository interfaces were placed inside the **domain** module in the **.repository** package, while the concrete implementations were placed in the same module, under the packages **.repository.mongodb** and **.repository.mariadb**.

Another remark is that the repository interfaces are implementation agnostic, as I've defined custom exception for error cases (duplication of department or doctor, overlapped shifts, and doctor, department, or shift not found), meaning that the interface methods declare these exceptions in `throws`, while the concrete implementations catch implementation specific exceptions, to throw the corresponding custom one.

Presenters

Presenters were implemented directly as concrete classes with four methods, `newEntity`, `removeEntity`, `updateEntity`, and `allEntities` (replace entity with either doctor, department, or shift). As it is just an orchestrator, each operation does the followin: calls doInTransaction on the transaction manager, using its provider to retrieve the necessary repositories, then perform the necessary operation(s)[3]. If the transaction doesn't throw an exception (it was committed), it will call the corresponding 'happy' method of the view, otherwise (transaction rollbacked) a `showError...` will be called on the view, depending on the thrown exception.

Presenters are placed inside the **presentation** module, in the **.presenter** package.

[3]For example, `newDoctor` just calls `save` on the DoctorRepository, while `removeDoctor` first removes all the shifts in the shift repository, before removing the doctor.

Views

Views were initially defined as interfaces with methods to show updates or the outcome of operations. I then implemented each as a Swing JPanel.

Each view (for each entity) requires to define the following methods: `showAllEntity`, `entityAdded`, `entityRemoved`, `entityUpdated`, `showErrorEntityNotFound` (replace entity with either doctor, department, or shift), and `showErrorDuplicateEntity` (for department and doctor, while shift has `showErrorOverlappedShifts`).

I chose to implement views as panels rather than frames, so that I could place them all together in a single frame for the final application (I decided to place each panel in a different tab of a JTabbedPane).

View interfaces were placed inside the **presentation**[4] module in the **.view** package, while the concrete implementations were placed in the same module, under the package **.view.swing**.

2.3 Transactions

In order to support transactions, I followed the example presented in the book, defining the interfaces `TransactionManager` and `TransactionCode<T>`. however, I expanded it (since I have three repositories rather than one), by defining the `RepositoryProvider` interface mentioned above, and defining transaction code to be `TransactionCode<T> extends Function<RepositoryProvider, T>`.

This allows presenters to pass a lambda to `doInTransactionCode` which takes a parameter of type repository provider (that allows the presenter to then work on the repositories). At that point, it was a matter of implementing the RepositoryProvider interface for both mongodb and mariadb (returning the corresponding concrete repository implementation), to wire the three repositories together (accessible from the same place). The actual transaction manager implementations are quite simple, and are talked about in the Technologies section below.

RepositoryProvider and its concrete implementations are placed in the same way as the repositories (stated above).

TransactionManager was placed inside the **domain** module in the **.transaction** package, while the concrete implementations were placed in the same module, under the packages **.transaction.mongodb** and **.transaction.mariadb**.

[4] I chose to place views and presenters together, rather than in their own module, since both presenters and views have a reference to the other. This could've caused a cyclic dependency import in the pom.xml files.

2.4 App module

The **app** module contains both the code for the main method and the end-to-end tests for the application. Inside the main `SwingHospitalApp` class, I defined various command-line options (through object fields, and by using the library Pico-cli) to allow users to define the preferred database (either MongoDB or MariaDB), which basically constructs the TransactionManager based on what database server it expects to work with, and also to define other database type-specific parameters, such as collection names for MongoDB or user/password for MariaDB.

3 Encountered problems and solution

During the development of this project, I mainly encountered the following problems: initial versions incompatibilities, flaky UI tests on Wayland (as expected), non-deterministic tests using LGoodDatePicker widgets, and mutation tests timeout.

3.1 Version incompatibilities

The project started out by setting the maven compiler version to `1.8`. Very early in development, between the creation of the concrete repositories, and the introduction of the transactional layer, I was using `mongo-java-server` to write 'unit' tests for the mongo implementations. However, in-memory databases (in particular this one) does not support transaction (or 'replica sets', as seen later). I initially considered just switching to testcontainers, however, I was also aware of some breaking changes brought by Docker daemon v.29, mainly because it started causing an `IllegalStateException: Could not find a valid Docker environment` error. A solution I found was just to increase the version of testcontainers to `1.21.4`, which required Java 17 minimum to compile. This also meant two things: firstly, I was now able to update many dependencies to newer versions, and secondly, I had to abandon the in-memory implementation tests, as the `mongo-java-server` uses a version of **Wire** protocol (used by Docker) not nearly close enough to the required one (2 compared to 8). As a result, both MongoDB and MariaDB repositories are tests directly as Integration Tests (since they require the database collaborator), even though I still applied TDD while writing those, meaning they present both happy and exceptional test cases, since I had to cover all possible paths.

This also meant taking a (personally) controversial choice, as repositories are considered Integration Tests, I put them under `src/it/java`, but I still wanted them to contribute to code coverage. I made an exception to the 'unit tests should be used to reach 100% CC' (or 'integration tests should not increase CC'), since these ITs are the only way that the repositories' code is tested. I excluded them from the JaCoCo **check** goal during the **test** phase, including them normally during the **check** goal of the **verify** phase. I thought about placing the tests under `src/test/java` since they were written as unit tests (since I used

TDD here to write tests first and code after), but I still chose the Integration Test source folder in the end.

3.2 Mutation tests timeout

Another tedious problem was a non-deterministic **WARNING: Minion exited abnormally, TIMED_OUT** problem.

This happened during mutation testing on `MariaDoctorRepository` (I only had implemented the `DoctorRepository` interface up until this point), where each mutant wasn't killed but neither survived, each just timed out after a while.

Since mutation testing executes tests for each generated mutants, I supposed that it had to do with the creation of a MariaDB container for each test suit (i.e. for each mutant). PIT measures the execution time of the tests before running them on the mutants, to set a threshold to use in case of infinite loops. I thought that the measured execution time of the tests on the original code could have made PIT set a threshold that not all other executions could suffice (e.g. the container could take more time to load when the host machine is overloaded).

I managed to solve this problem partially by increasing the **timeoutFactor** to `2.0` and, even though more mutants were killed, most still remained non deterministic. For that reason, I also decided to add a fixed `timeoutConstant` of `35s`, which is a little pessimistic but managed to make mutation testing deterministic for this class.

In the end, I thought about excluding the repositories from mutation testing altogether because of the extended required build time. However, since I managed to overcome that issue, I preferred to keep PIT configured in the `pluginManagement` without the repositories and define two profiles: `mutation-testing`, which just enables the plugin, and `mutation-testing-full`, which also overrides the configuration to include the repositories.

Note that the `TransactionManager`s (both implementation) were not included in mutation testing, as they just wrap the given `TransactionCode` in the corresponding transactio API calls. Mutating here could cause problems such as memory leaks or dangling connections (suppose a mutator removes the `close` on an `EntityManager`, or `rollback`). On the other hand, excluding these methods was not be an option either, as they are the only instructions performed, and would lead to mutate almost nothing.

3.3 Wayland UI tests

As already stated in the course UI tests on Wayland are flaky because of how `java.awt.Robot` class works, as it simulates user input by moving the cursor and clicking, or typing keystrokes. While this tends to work fine on X11, the same doesn't hold for Wayland, as it enforces a stricter mouse and keyboard security mechanism, and `Robot` cannot just 'take control of

it' (also, X11 apps don't run natively on Wayland, but on a compatibility layer, XWayland).

I fixed this problem locally by using `Xephyr` (an X11 virtual server, as `Xvfb`, but actually creates a visible display) in combination with `openbox`. Xephyr by itself wasn't enough as the window inside the display sometimes lost focus (this happened for examples in the presence of pc notifications). The solution was to add a window manager to Xephyr (e.g. launching openbox), so it can keep the window focused, as well as keeping track of focused widgets.

With this setup I was able to deterministically run tests locally (I still needed to 'lock mouse and keyboard' within Xephyr, which meant that any other external program wouldn't interfere) using `mvn` from the terminal. In order to run the same tests on Eclipse I had to modify the Run configuration by setting an environment variable `DISPLAY`, equal to `:50` (it was the Xephyr's display number that I set)[5].

On GitHub actions, I initially fixed this by running `xvfb-run -a mvn` instead and, while this worked initially, as I created the `SwingShiftView` GUI, this method started to be non-deterministic as, sometimes, the Window would be too big for the virtual display (and components would be out of reach). I tried setting a fixed display screen, which worked initially, but still remained non deterministic, since it sometimes lost focus of the window. The solution, once again, was to use openbox here as well, to add a window manager to the X11 server.

Usage example

First I create a Xephyr instance with `Xephyr :50 -screen 600x800 &` (it's a sufficient resolution). This creates a visible X11 display. After that I run `DISPLAY=:50 openbox &` to start the window manager on the display created by Xephyr. At this point, I could run `DISPLAY=:50 mvn verify` to run all the tests on the Xephyr's display. Alternatively, I could open eclipse, go to 'Run As' → 'Run configurations' and create a new configuration for any test suite with `Robot` that we want to execute, and set an environment variable as in figure 3. Now, running a test that opens a GUI will have create windows inside the virtual

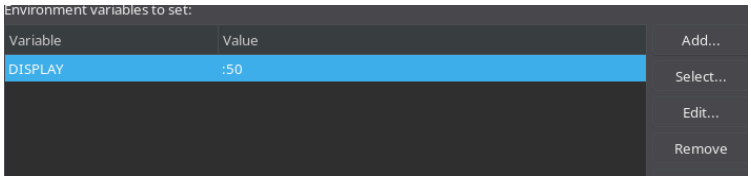


Figure 2: Custom environment variable for JUnit Run Configuration

[5]I also tried `xvfb-run` instead, which works with the same procedure (running openbox with its display number). I noted that while on Xephyr the mouse and keyboard need to be kept still to not interfere, on xvfb-run, we can also set an environment variable `NO_AT_BRIDGE=1`, which completely separates xvfb's mouse and keyboard from the host's ones, so one can keep using its PC.

display, and by locking the mouse and keyboard (it should be `ctrl-shift`) the tests will run and pass, deterministically.

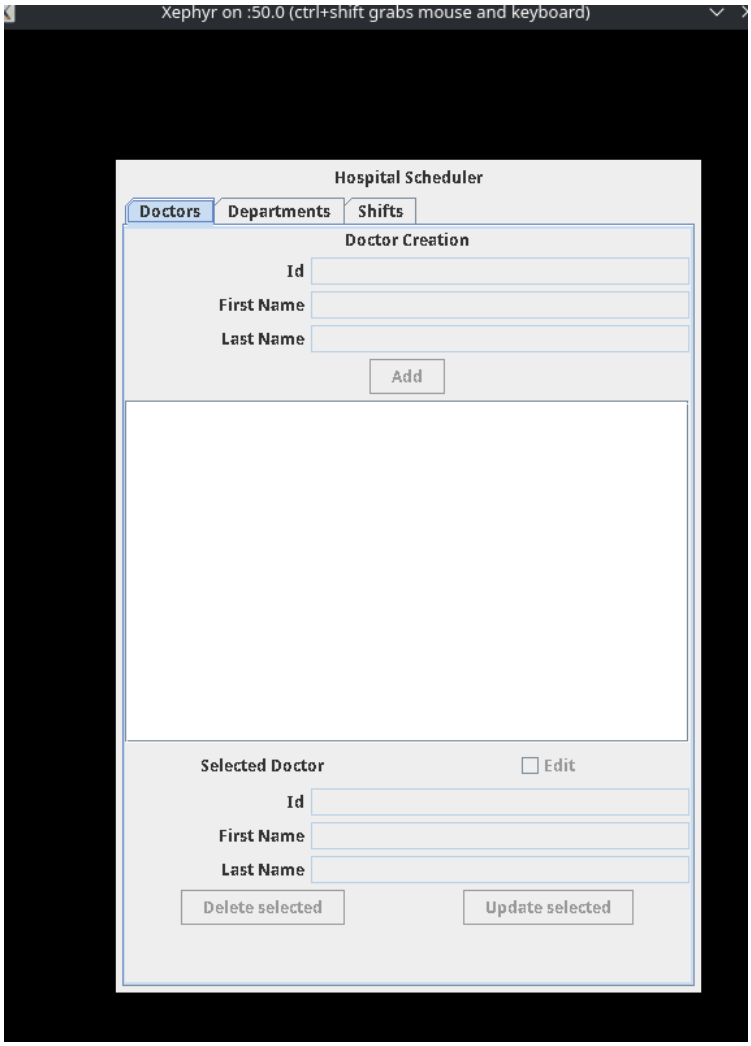


Figure 3: Window being created within Xephyr

3.4 LGoodDatePicker non-determinism in UI tests

While creating the `ShiftSwingView` GUI (this technology is better explained later) I decided to use **LGoodDatePicker** to add both Date and Time pickers (since Shift uses both `LocalDate` and `LocalTime`). Briefly, each widget has a textfield and a button, which opens either a calendar or a list of times (for Date and Time respectively). While testing 'Add' button enabling logic in `SwingShiftViewTest`, I wanted to consider both input methods valid towards the its activation. The text field was no problem, as it can be given a name

to access it through `window.textBox(..)`. Same goes for clicking the button, through `window.button(..)` and passing its name (again, it can be set). However, the problem came after clicking the button, as the components within both the calendar and the time list have no names.

I was able to find a solution after some tests failing with `ComponentLookupException`, since it prints the component hierarchy. I noticed the clickable days in the calendars where simple `JLabel`s, which made it possible to obtain its Fixture by using a `GenericTypeMatcher <JLabel>`. However, this already is not particularly simple or reliable, as adding just one label with a number within 1 and 31 in the main GUI will make this function not work. Moreover, when the calendar is open, the month is set to the current one, which means that the day will be chosen from the current month. I just kept it to make sure user input from the calendar works towards enabling the button, so I chose to use a day present in each month (i.e. < 30).
A similar approach was taken for the TimePicker, its button opens a `JList`. The previous approach wasn't particularly pleasant, as I should've used a `GenericTypeMatcher<JList>`, while JList is parameterized, which gave a warning. I could also use a wildcard ? notation, but would've required an explicit cast. In the end, I decided to use a `GenericTypeMatcher<Component>`, comparing the class with `getClass().equals(JList.class)`. Not that I really like this workaround, it made it possible to retrieve a component that was a JList, and I used it to create a JListFixture that could be used by the robot to click the time.

3.5 Development environment

The project was entirely developed on my local machine, running the following:

- Operating System: CachyOS (Linux kernel version 7.2.2-1-cachyos);
- Desktop Environment: KDE Plasma 6;
- Window Compositor: Wayland;
- Java version OpenJDK 17;
- Workarounds for UI tests: both xvfb-run and Xephyr paired with openbox.

Of course I also used a CI (GitHub Actions) environment, which also helped in verifying that tests didn't just 'work locally'. In both workflows, I started from an empty 'ubuntu-latest', installing both `xvfb` and `openbox`, while also pulling the container images beforehand, as I noticed the workflow would sometimes fail with a timeout, which I assume was because of the on-the-fly image pulling. Pulling beforehand fixed this deterministically.

4 Used technologies

The main technologies used for application development were the following: TDD, JUnit 5, AssertJ, AssertJ Swing, Maven (with Maven Wrapper), JaCoCo, Mockito, PIT, Git (and GitHub), CI (with GitHub Actions), Docker, MongoDB and MariaDB (both with Testcontainers), Coveralls, SonarQube (mainly SonarCloud), Log4J, Pico-cli, EqualsVerifier, JPA (with Hibernate), Awaitility, and LGoodDatePicker.

As for the technologies used to offer a simple build and execution mechanism there are Make, Docker Compose, Xhost, Xvfb (and xvfb-run) and Openbox.

4.1 High rating technologies

Amongst the previously mentioned technologies, each of the following falls within the category of 'high rating project': MariaDB, JPA, Hibernate, MongoDB transactions, and DB selection.

MariaDB

MariaDB was chosen to offer a second implementation for the repositories (since the main implementation is in MongoDB) that relied on a SQL database. Although I didn't work with MariaDB connections and raw queries directly in the Java code, I have mainly used MariaDB in tests (by using a MariaDB testcontainer) and in production, by giving two *docker-compose.maria.yml* files, used by `make` to start the application either natively or within a Docker container.

As a matter of fact, I mostly worked with MariaDB through the library **Hibernate** in the code.

JPA

To have a more easy approach with an SQL database, I chose to use an ORM library. ORM stands for 'Object Relational Mapping' and refers to a way to transale Java Objects to SQL table records (and vice versa). JPA is a standard that defines ORM management through the use of tags and interfaces. There are many tags to be used in Java classes (the Model entities' classes) such as `Entity` on the class name, to mark a class as a model entity, `Table(name = "..")` on the class name, to set the entity's table name, `Id` on a field, to set the primary key of the entity, and `Column(name = "..")` on a field to specify the name of the corresponding column in the database[6]. An entity class needs to be `public` and have the following: a public no args constructor, a public constructor with all args, public getters for each field, and public setters for each field.

[6]There are many other tags, most of them I didn't really see. The ones that I listed were the ones sufficient for the project.

Since I wanted to keep my model classes implementation agnostic, I choose to create, within the package `org.duckdns.whocaresleft.repository.mariadb`, a package `entity` with three classes: `DoctorEntity`, `DepartmentEntity`, and `ShiftEntity`. These have the same fields [7] as the 'original' counterpart, while meeting the JPA `Entity` requirements, so I created two helper methods to map each model object to its 'Entity' (JPA) counterpart, and vice-versa, so I could keep using my model objects in the other classes, without them being JPA-specific.

JPA requires a **persistence.xml** file (inside **META-INF/persistence.xml**, usually in the main resource folder). This file defines 'persistence units' (aggregation of Entity(s)), each of which defines the name of the unit, the ORM provider (**Hibernate** was used in this project), a list of class entities (Entity), and other database connection properties or configurations (such as username or password for instance). At runtime, a persistence unit is managed by an `EntityManagerFactory` (or EMF for short). The latter requires the name of a persistence unit, at creation, but also accepts a `Map<String, String>` of 'properties', which can be used to override the configuration of the file persistence unit in the **persistence.xml** file.

The EMF is a thread safe object, shared between all threads that work on the same persistence unit. The actual DB operations are performed with a lighter, non thread safe, `EntityManager` (or EM). As the name suggests, the EMF is used to create EM(s), the former are the ones that need to be used to perform CRUD operations on the db. These are performed in an object-oriented manner, through the methods `persist(Entity)`, `find(EntityClass, Id)`, `merge(Entity)`, and `remove(Entity)` respectively. Every operation performed on the EM needs to happen within a transition, as the EM works with an inner memory (called L1 cache), updates on the database are only performed when `getTransaction().commit()` is explicitly called on the EM (this requires `getTransaction().begin()` before starting). This aspect impled two things, the first one was that the application now required the use of transactions (since these are required for JPA) even for MongoDB, while the second one was that in JPA, they are overly simple (`begin` followed by either `commit` or `abort`, while code is purely object oriented, with no raw queries[8])

The `MariaTransactionManager` just wraps the given `TransactionCode` in a JPA transaction. It starts it with `begin`, executes it, and tries to `commit` inside a `try-catch` block. If any exception is thrown in the meantime (we recall that custom Model error situation such as duplication or entity not found, are wrapped in custom exceptions) it will abort the

[7] Apart from the `Id`, which is just a string here. However, considering these objects are only instantiated by converting valid model objects, or from the database, these should be considered valid. The only scenario in which it wouldn't is if someone inserted an entity with a non-valid id (but valid string) in the database directly.

[8] Well, just a few, because there isn't a `findAll`, but selecting everything from a table is one of the simplest SQL queries.

transaction with `rollback`.

Hibernate

Hibernate is a library that works as an ORM provider, also completely compatible with JPA (even though it could do more). While JPA is a specification, Hibernate is the actual ORM provider (recall that in the persistence.xml file we need to specify which ORM provider to use). In practice, I've used Hibernate only at runtime as an ORM provider, while writing pure standard JPA code.

MongoDB transactions

As previously stated, both in sections design choices, and JPA tecnhology, the application presenters rely on transactions. These were fairly simple to implement for the `MariaTransactionManager`, and so were for `MongoTransactionManager` surprisingly.

Transactions in MongoDB have some requirements, both code wise and container wise. From a code point of view, this requires creating an object of type `ClientSession` from the `MongoClient` to be used in every operation that is part of the same transaction (or, as mongodb calls it, session). Repository implementations change very little from a non-transaction mongo implementation, as the repository just possesses the `ClientSession`, and passes it to every operation method[9]. This was possible thanks to the `RepositoryProvider` design choice, which meant that the repository only lives for the transaction it is created in (the session is valid for the entire repository's life).

The final touch was implementing the `MongoTransactionManager`, which creates a `ClientSession` on the fly, gives it to the repository provider (which will forward it to the repositories) and execute the given transaction code inside a transaction with the `ClientSession` method `doInTransaction` which automatically commits if no exceptions are thrown, and rollbacks otherwise.

From a container stand point, MongoDB requires a configured replica set in order to support transactions. For tests this was implicitly done, since Testcontainers already set a replica up (which I connect to using `getReplicaSetUrl()`). On the contrary, the replica set was explicitly configured in the **docker-compose.mongo.yml** files. The replica set is a collection of nodes, generally it aims for at least three nodes (primary, secondary or backup, and a failerover). I just configured a single node replica set (just one node), not really the best solution for production, but it was enough for the scope and simplicity of this application. In the end, the replica set configuration can be done in javascript within the docker compose file, with two function calls and a conditional: we first call `rs.status()` and check if the

[9] We can still call methods without passing the session, but they will be executed outside the transaction, so a rollback won't revert those.

node is already part of a replica set. If not, we configure a new replica set called **rs0** with just one node, using `rs.initiate(..)`.

Database selection at launch

Since I implemented the transactional and data layers to work with two possible different databases, I also wanted to offer the decision to choose whatever at launch (i.e. when launching the **.jar**). The library Pico-cli was used for this goal, since it allowed to define and parse command line arguments in a much simpler way. The main argument in this scenario is `--db-backend`, which can either be `mongodb` or `mariadb` (default is mongodb, also lower or upper case does not matter). The main class defines a `getTransactionManager` method, which handles the implementation decision, based on the given parameter, while also making the `call()` method pretty clean and implementation agnostic.

The method, based on the parameter's value, either creates a `MongoTransactionManager` or a `MariaTransactionManager`, using the other CLI argument accordingly, which means:

- Using `--mongo-connection-string`, `--db-mongo-name`, `--db-mongo-doctor-collection`, `--db-mongo-department-collection`, and `--db-mongo-shift-collection` to create the MongoDB client and transaction manager;
- Using `--maria-jdbc-url`, `--maria-user`, `--maria-password`, and `--maria-ddl` to define the properties, define the EMF, and the transaction manager[10].

There is also a `--connection-timeout` that can be set, which is as a threshold to wait before considering either database unreachable and terminating the application. The default value is 15 seconds.

4.2 Other new technologies

This project also relies on technologies that were not seen during the course, other than the previously mentioned ones. These are EqualsVerifier, Awaitility (in part), and LGoodDatePicker.

EqualsVerifier

EqualsVerifier is a very simple library with one purpose, making testing `equals` and `hashCode` not cumbersome. I have used EqualsVerifier in two classes, Id and Shift. Even though the first one is just a string wrapper, and the second one is a model object, I created a test class for both for the following reason: they contain domain logic. Id requires the inner string to contain only letters, digits, and underscores, while shift requires both Ids to be not null,

[10]Note that it is not required to set the database or tables names, as these are already handled in the persistence unit.

the two dates to be consistent with one another, and to implement logic to check if a given shift overlaps with another one.

Since I included these in Code Coverage reports, I had to cover also `hashCode, equald,` and `toString`. On one hand `toString` actually made sense, since I wanted custom behaviour for both: toString of Id just prints the inner string, while toString of Shift prints a custom format of its values. However, on the other hand, checks for equals and hashCode require to cover many cases, which while being very simple, end up being too cumbersome.

I found this library as a viable solution, as it can test both hashCode and equals' contracts in one single test case (one line, actually).

The one liner is `EqualsVerifier.forClass(<className>.class).verify();` and generally it would be used when equals or hashCode are implemented differently than a possible default implementation.

Awaitility

Awaitility is a Java DSL for asynchronous operation synchronization. I have used this library in the same way as in the chapter 'Multithreading' of the book, since I chose to implement the presenter's methods called from new views in a separate thread. For that reason, the main usage of this library is in tests where a View is either the SUT (or part of it).

LGoodDatePicker

LGoodDatePicker is a small library that adds Swing components to easily pick dates and times. I found this library during the development of the SwingShiftView, since I needed a reliable way of picking both dates and times, before trying to implement it from scratch.

This library offers a `DatePicker` which is a component with a textfield (which is automatically validated on writes), and a button that opens a calendar. Another component that I used is the `TimePicker`, which is made up of a textfield (yet again, automatically validated on writes), and a button that opens a JSpinner (list with options) with a list of times (the start, end, and step can be configured after creation).

These were easily incorporated in the initial GUI I already had created, while also allowing me to keep the UI tests very similar to the ones I already had for SwingDoctorView and SwingDepartmentView (which are similar to the ones made during the course), as the text field can be given a name, so text can be entered, and checked, through `window.textBox(..)`. The only downsight that I encountered with this library, is that the calendar (and time) labels, are not named, nor can be named. This made it particularly

difficult to write tests that interacted with the button, instead of the textfield. However I was able to find two solutions:

- The first one is that both components already offer a listener for when a new valid date or time is set, whether it was written directly into the textfield, or chosen through the menu;
- Secondly, I was still able to use a GenericTypeMatcher for both components to access either the 'day number' label in the calendar and the list of times. However this was possible after various errors and attempts, by analyzing the stack trace to understand the components hierarchy[11].

5 Building instructions

The application can be built from the project's root directory by using the given Maven wrapper (specifying `-f hospital-scheduler-aggregator/pom.xml` and enabling the desired profiles (usually `JaCoCo` or `Pitest`). However, the preferred way to build and test the application is to use `make`. I have tested building only on Linux, specifically because of the tests that relied on `assertJ-Swing`. More information will be given later as instructions or tips. In any case, the project requires the following building dependencies: `Java` (JDK, at least 17), `Docker`, `xvfb-run`, and `openbox`[12].

5.1 Building in Linux

Even though I highly recommend using `make`, if one wishes to not use it, here are the steps to build and test the application, starting from the subfolder **hospital-scheduler-aggregator**. Use `./mvnw clean` and add the desired maven phase: `test` to run unit tests only, `package` to build the FatJar after unit tests, or `verify`, to also run integration (and ent-to-end) tests right after. The following profiles can also be used:

- `jacoco`, to generate the coverage report (done in phase `verify`), which can be later found in the module `hospital-scheduler-report`;
- `mutation-testing`, to also run mutation testing, with **STRONGER** mutators, on classes **Id**, **Shift**, and the **Presenters**;
- `mutation-testing-full`, which, in addition to the previous one, also run mutation tests on the repository implementations[13].

[11] For example, I noted that the times were stored in a JList with a null name, in the end I had to retrieve the list by matching a component that was a JList, was visible, and had null name. This was extracted into a helper method, so the test is readable, and I should be calling it in one or two tests only.

[12] The **README.md** file on the GitHub repository lists the package names for the two 'mainstream' package managers, `apt` and `pacman`.

[13] Note that this also increases the build time by around 5 to 10 minutes, that is, the time I have noticed both on my local machines and on GitHub actions.

Using make

Alternatively, `make` can be used directly on the project's root folder. There is only a single command that is purely for building (just compiling the FatJar, no test whatsoever), and that is `make package-only`, which just compiles the project and creates the FatJar.

As for the other commands, consider that since they run tests (specifically, also UI tests), for the maximum reproducibility on Linux, I opted to use both `xvfb-run` and `openbox`. As a matter of fact, `make` was used to automatically wrap `mvn` commands as follows: `xvfb-run -a -s '-screen 1920x1080x24' bash -c "openbox & ./mvnw <maven phases> -f hospital-scheduler-aggregator/pom.xml`[14]. `xvfb-run` creates a virtual display entirely in memory, the `a` flag searches automatically for an available display number, while `s` sets a screen size (apparently the default one could not be enough to fit the application, resulting in components out of reach from the display). `Openbox` is an X11 window manager, which handles the frames 'drawn' on `xvfb`, without it, tests failed non deterministically because windows and widgets inside `xvfb` lost focus. The commands are the following (each also executes `clean` beforehand):

- `make package`, which executes the **package** phase of Maven;
- `make run-coverage`, which executes **verify**, with the `jacoco` profile activated;
- `make run-mutation-light`, which executes **verify**, with the `mutation-testing` profile activated;
- `make run-mutation-full`, which executes **verify**, with the `mutation-testing-full` profile activated;
- `make run-all`, which executes **verify**, with both the `jacoco` and `mutation-testing-full` profiles activated.

Running tests in Eclipse

Before running the test in Eclipse, if the projects aren't already imported in Eclipse, one can do so by navigating to **'File → Import → Maven → Existing Maven Projects → Browse'**, from the file dialog, search for the root folder (which should be **hospital-scheduler**) and press 'Open'. This should ask to import all seven modules of the project.

Inside the **hospital-scheduler-aggregator** there are three launch options, since the project is multi module, all tests cannot be launched with just one **'Run as → JUnit Test'**. However, this can be done by using the latter method on the three modules that do contain tests: **app** (E2E tests), **domain** (UT and IT for repositories and transaction manager), and **presentation** (UT and IT for presenters and views). Please note that the

[14] Actually, it also removes some Wayland specific envinronment variables, which caused problems during tests execution, and now it should completely isolate the local mouse and keyboard from the virtual ones (i.e. the user CAN use their computer, if they really want to).

tests in **presentation** and **app** mostly rely on **AssertJ Swing** since they are testing an application with a GUI, so make sure that you are using X11. As an alternative, if using Wayland, one can use either `Xvfb` or `Xephyr` with `openbox`. In such case, one would need to run either `Xvfb :99 -screen 0 1920x1080x24 &` or `Xephyr :99 -screen 1920x1080 &` (display number and resolution could be changed) followed by `DISPLAY=:99 openbox &`. At that point, one would need to modify the desired launch configuration, by navigating to the 'Environment' tab and add a variable called 'DISPLAY' with a value of ':99'. At that point, running the launch configuration should have tests running on the virtual display. Otherwise, just use X11, or the `make` commands, which already take care of this.

6 Running instruction

The application can be run in two different ways, being natively on your local machine or within a Docker container. In both cases, the use of `make` is highly recommended.

6.1 Running natively

In order to run the application directly on the host machine, all *Building dependencies* 5 are required. In any way, executing the next commands will first print the dependencies. When executing the `make` command, one can choose the preferred database, as for now, between **MongoDB** and **MariaDB**. These options need to be passed to `make`:

- `make native-build-and-run DB=mongo` to use **MongoDB**;
- `make native-build-and-run DB=maria` to use **MariaDB**;
- or alternatively just `make native-build-and-run`, which falls back on **MongoDB**.

This command builds the application creating a FatJar, then it starts a Docker container with the corresponding database, waiting for it to finish the healthcheck. Once that is done, it starts the application natively (through `java -jar`). Once finished, the application can be closed simply by pressing the X button of the native window. Finally, the container should be stopped, and to do so, either one of the following commands can be used:

- `make native-stop` to stop the container while keeping the volumes, meaning that the next executions will have the persisted data;
- `make native-clean` which removes the volumes in addition to stopping the container.

6.2 Running in Docker (Linux only)

Alternatively, one can choose to run the application itself from a Docker container, although this is available only on Linux, where both `docker compose` and `xhost` need to be

installed [15].

Similarly to before, the preferred database can be chosen, one can use one of the following command variants: database, as for now, between **MongoDB** and **MariaDB**. These options need to be passed to `make`:

- `make docker-build-and-run DB=mongo` to use **MongoDB**;
- `make docker-build-and-run DB=maria` to use **MariaDB**;
- or alternatively just `make Docker-build-and-run`, which falls back on **MongoDB**.

Either variant will start two containers, one with the database and one with the application. The latter will copy the source project files, build the FatJar, and execute it. The execution of a GUI application run by a Docker container was achieved by forwarding the X11 server to the container (by setting the IPC X11 socket in the Docker compose) while also granting the local user access to said server (using `xhost`). Similarly to before, once finished, the application can be closed simply by pressing the X button of the native window. The containers should be stopped as well, and to do so, either one of the following commands can be used:

- `make docker-stop`, which stops the containers while keeping the volumes, meaning that the next executions will have the persisted data;
- `make docker-clean`, which removes the volumes in addition to stopping the containers.

6.3 Manual execution

If one wishes to start the application manually, they can do so just by using `java -jar`. Note that this requires a running database (properly configured with a replica set in case of MongoDB). When running the application, recall that cli arguments can be given to configure both the database type and connection, consider the image 4 as reference.

[15] The **README.md** in the GitHub repository also contains the package names of these dependencies for both 'mainstream' package managers, `apt` and `pacman`.

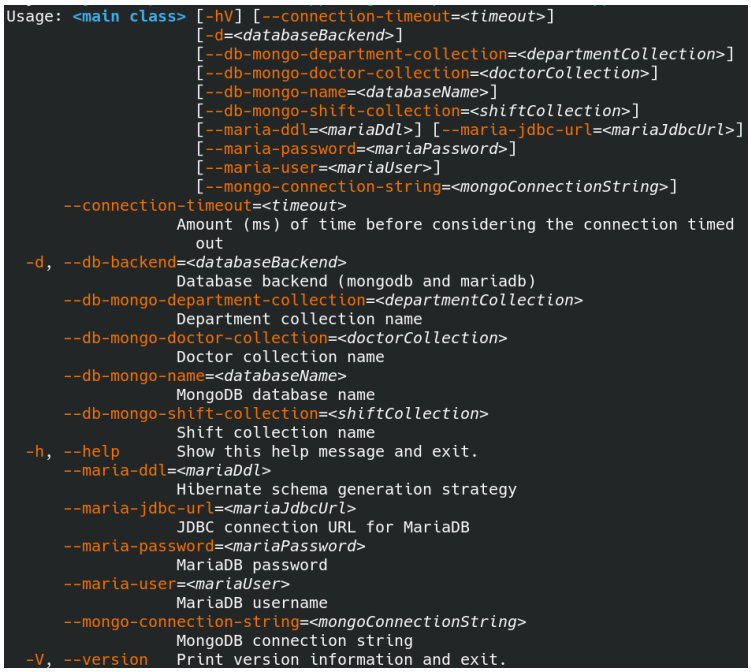


Figure 4: CLI arguments for SwingHospitalApp